

MePaga - Documentação Completa Consolidada

Versão: 1.0.0

Data: 2026-06-11

Status: ☒ Completo

Índice Geral

1. [Guia Inicial \(COMECE AQUI\)](#)
2. [README - Visão Geral](#)
3. [ARQUITETURA - Técnico Completo](#)
4. [REQUISITOS - Análise Completa](#)
5. [DOCUMENTACAO - Guia Central](#)
6. [QUICK REFERENCE - Cheat Sheet](#)

GUIA INICIAL

COMECE AQUI - Índice de Documentação MePaga

Bem-vindo à documentação do projeto **MePaga**! Este documento ajuda você a encontrar exatamente o que precisa.

Documentação Disponível

Foram criados **5 documentos principais** com ~83 KB de conteúdo detalhado:

README.md (11 KB) - INÍCIO AQUI

O que é: Visão geral do projeto

Para quem: Todos (novo no projeto?)

Conteúdo:

- O que é MePaga
- Quick Start (como instalar)
- Stack tecnológico
- Funcionalidades principais
- Estrutura do projeto
- FAQ

ARQUITETURA.md (26 KB) - MAIS COMPLETO

O que é: Documentação técnica detalhada

Para quem: Desenvolvedores, arquitetos

Conteúdo:

- Padrão Clean Architecture
- Estrutura de 5 camadas
- Banco de dados (schema Prisma)
- Endpoints da API completos
- Fluxos de dados
- Integração frontend-backend
- Segurança
- Melhorias futuras

REQUISITOS.md (22 KB) - ESPECIFICAÇÕES

O que é: Análise completa de requisitos

Para quem: Product Managers, Analistas, Testadores

Conteúdo:

- 14 requisitos funcionais detalhados
- 7 categorias de requisitos não-funcionais
- 5 casos de uso principais
- 5 user stories
- Critérios de aceitação

- Roadmap futuro
- Matriz de rastreamento

4 DOCUMENTACAO.md (12 KB) - GUIA CENTRAL

O que é: Índice e guia de navegação

Para quem: Todos

Conteúdo:

- Como usar a documentação
- Fluxos de aprendizado por perfil
- Resumo de arquitetura
- Estrutura de camadas
- Entidades principais
- Fluxos principais
- Métricas de qualidade

5 QUICK_REFERENCE.md (12 KB) - CHEAT SHEET

O que é: Referência rápida para desenvolvimento diário

Para quem: Desenvolvedores em manutenção

Conteúdo:

- Comandos essenciais
- Estrutura de pastas
- Padrão de código
- Snippets de banco de dados
- Endpoints (tabelas)
- Testes rápidos com Postman
- Debug útil
- Erros comuns

Guia por Perfil

Desenvolvedor Novo no Projeto

Dia 1:

1. Leia este guia - 5 minutos
2. Execute "Quick Start" - 10 minutos
3. Explore a estrutura local - 10 minutos

Dia 2:

1. Leia DOCUMENTACAO seção 1-3 - 15 minutos
2. Leia ARQUITETURA seção 1-4 - 20 minutos
3. Explore código seguindo a estrutura - 30 minutos

Dia 3+:

1. Leia ARQUITETURA completo
2. Estude QUICK_REFERENCE para reference
3. Comece a fazer mudanças pequenas

Product Manager / Stakeholder

Reunião 1:

- README - Visão geral do projeto
- Entender: O que é, stack, funcionalidades

Reunião 2:

- REQUISITOS seção 1-3 - Funcionalidades
- REQUISITOS seção 9 - Roadmap

Reunião 3+:

- REQUISITOS seção 10 - Matriz de rastreamento
- Usar para acompanhar progresso

Testador / QA

Preparação:

1. README - Entender projeto
2. REQUISITOS seção 2 - Funcionalidades
3. REQUISITOS seção 2 (critérios de aceitação)

Durante Testes:

- QUICK_REFERENCE - Testar endpoints
- ARQUITETURA seção 6 - Fluxos de dados

Arquiteto / Tech Lead

Visão Completa:

1. ARQUITETURA - Estrutura técnica
2. REQUISITOS seção 3-4 - Requisitos não-funcionais
3. DOCUMENTACAO - Métricas

README - VISÃO GERAL

MePaga - Divisão de Despesas

Um aplicativo moderno para simplificar o gerenciamento de despesas compartilhadas entre grupos de amigos, colegas e familiares.

O que é MePaga?

MePaga permite que você:

- ☒ Crie grupos para organizar despesas
- ☒ Registre despesas compartilhadas automaticamente
- ☒ Veja exatamente quem deve quanto para quem
- ☒ Compartilhe chaves PIX para transferências fáceis
- ☒ Mantenha histórico completo de transações
- ☒ Use em iOS, Android e Web

Arquitetura do Projeto

MePaga é composto por duas aplicações principais:

1. Backend (API REST)

- **Framework:** Fastify
- **Banco de Dados:** PostgreSQL
- **ORM:** Prisma
- **Padrão:** Clean Architecture
- **Localização:** `mepaga-api/`

2. Frontend (Aplicativo Mobile)

- **Framework:** React Native
- **Plataforma:** Expo
- **Navegação:** React Navigation
- **Suporte:** iOS, Android, Web
- **Localização:** `mepaga-app/`

Quick Start

Requisitos

- Node.js 18+
- PostgreSQL 12+
- pnpm (ou npm)

Instalação - Backend:

```
cd mepaga-api
pnpm install
pnpm prisma:migrate
pnpm dev
# Acesso em http://localhost:3000
```

Instalação - Frontend:

```
cd mepaga-app
pnpm install
pnpm start
# Escolha: Android, iOS ou Web
```

Funcionalidades Principais

☒ Implementadas

Autenticação

- Registro de novo usuário
- Login com email/senha
- Recuperação de senha (código por email)
- Atualização de chave PIX

Grupos

- Criar grupos (com categorias)
- Entrar via token de convite
- Visualizar membros do grupo
- Deletar grupo (criador)
- Histórico de atividades

Despesas

- Criar despesa compartilhada
- Divisão automática entre membros
- Upload de recibo/foto
- Visualizar histórico

Saldos

- Cálculo automático de quem deve para quem
- Visualizar chave PIX de quem receber
- Resumo financeiro do usuário
- Breakdown por grupo

Estrutura do Projeto

```

mepaga/
├── mepaga-api/
│   ├── src/
│   │   ├── domain/           # Entidades e interfaces
│   │   ├── application/      # Use cases
│   │   ├── infrastructure/    # Repositórios
│   │   ├── presentation/     # Controllers e rotas
│   │   ├── shared/           # Erros, utils
│   │   ├── app.ts            # Config Fastify
│   │   └── server.ts         # Entry point
│   ├── prisma/
│   │   └── schema.prisma     # Schema do banco
│   ├── package.json
│   └── tsconfig.json
├── mepaga-app/
│   ├── src/
│   │   ├── screens/          # 16 telas da app
│   │   ├── components/       # Componentes reutilizáveis
│   │   ├── icons/            # Ícones SVG
│   │   ├── services/         # Client HTTP
│   │   ├── navigation.ts     # Tipos de navegação
│   │   ├── theme.ts          # Temas e estilos
│   │   └── App.tsx           # Componente raiz
│   ├── app.json              # Config Expo
│   ├── package.json
│   └── tsconfig.json
├── ARQUITETURA.md           # Documentação técnica
├── REQUISITOS.md             # Análise de requisitos
├── DOCUMENTACAO.md           # Índice e guia
├── QUICK_REFERENCE.md        # Referência rápida
└── README.md                 # Este arquivo

```

Modelo de Dados

Entidades Principais

User - Usuário da aplicação

- Autenticação com email/senha
- Chave PIX para recebimentos
- Múltiplos grupos

Group - Grupo de despesas compartilhadas

- Token único de convite
- Múltiplos membros
- Histórico de despesas

Expense - Despesa registrada

- Valor total
- Quem pagou
- Divisão entre membros

ExpenseSplit - Divisão de despesa

- Quanto cada membro deve
- Referência a expense e user

Endpoints da API

Autenticação

```

POST  /auth/register
POST  /auth/login
POST  /auth/forgot-password
POST  /auth/reset-password

```

Grupos

```
POST    /groups
POST    /groups/join
GET     /groups/:groupId/invite
GET     /users/:userId/groups
DELETE  /groups/:groupId
```

Despesas

```
POST    /expenses
GET     /groups/:groupId/activities
GET     /users/:userId/activities
```

Saldos

```
GET     /groups/:groupId/balances
PATCH  /users/:userId/pix-key
GET     /users/:userId/summary
```

Stack Tecnológico

Backend

- **Fastify** 5.8.5 - Framework web rápido
- **TypeScript** 6.0.3 - Linguagem tipada
- **Prisma** 7.8.0 - ORM moderno
- **PostgreSQL** - Banco de dados
- **bcrypt** - Hashing de senha
- **Node.js** - Runtime

Frontend

- **React Native** 0.85.3 - Framework mobile
- **Expo** 56.0.7 - Plataforma
- **React** 19.2.3 - UI library
- **TypeScript** 6.0.3 - Linguagem tipada
- **React Navigation** 7.2.5 - Navegação

Segurança

- ☒ Senhas com hash bcrypt (10 rounds)
- ☒ Validação rigorosa de entrada
- ☒ CORS configurado
- ☒ Autorização por usuário
- ☒ Proteção contra SQL Injection (Prisma)
- ☒ HTTPS em produção

ARQUITETURA - TÉCNICO COMPLETO

1. Visão Geral do Projeto

1.1 Objetivo

MePaga permite que grupos de usuários dividam despesas de forma simples e eficiente. A aplicação gerencia:

- Criação e participação em grupos
- Registro de despesas compartilhadas
- Divisão automática de custos
- Controle de débitos e créditos entre membros
- Sincronização de dados em tempo real

1.2 Tecnologias Principais

Backend:

- **Fastify** v5.8.5 - Framework web rápido e eficiente
- **TypeScript** v6.0.3 - Linguagem tipada
- **Prisma** v7.8.0 - ORM moderno com migrações
- **PostgreSQL** - Banco de dados relacional

- **Node.js** - Runtime JavaScript server-side

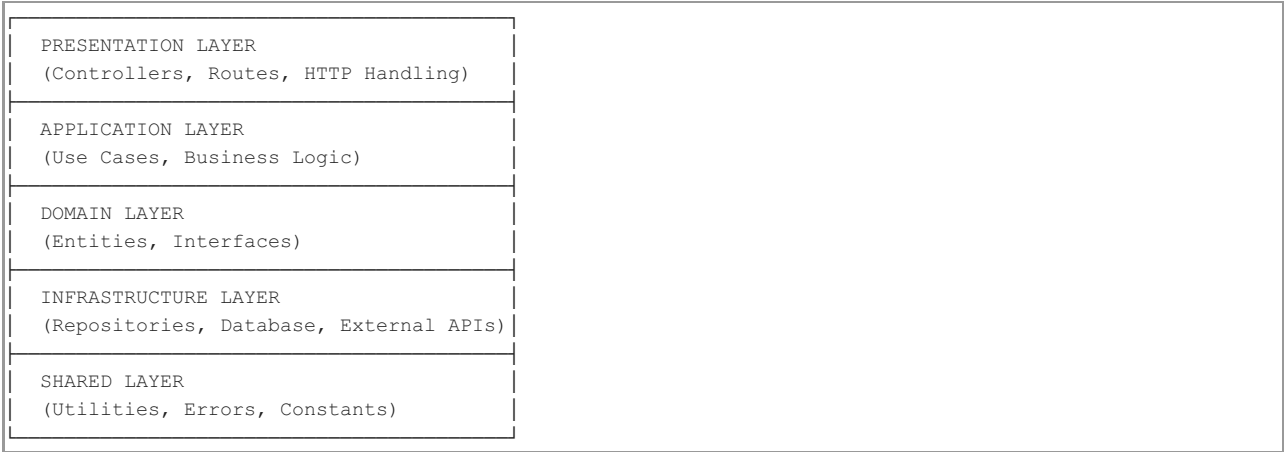
Frontend:

- **React Native** v0.85.3 - Framework cross-platform
- **Expo** v56.0.7 - Plataforma de desenvolvimento
- **React** v19.2.3 - Biblioteca UI
- **TypeScript** v6.0.3 - Linguagem tipada
- **React Navigation** v7.2.5 - Navegação entre telas

2. Arquitetura da API (Backend)

2.1 Padrão Arquitetural

A API segue **Clean Architecture** com separação em camadas bem definidas:



2.2 Estrutura de Diretórios

```

src/
├── domain/                                # Regras de negócio
│   ├── entities/                         # Modelos de dados
│   │   ├── User.ts
│   │   ├── Group.ts
│   │   ├── Expense.ts
│   │   └── Balance.ts
│   ├── repositories/                    # Interfaces de repositórios
│   │   ├── IUserRepository.ts
│   │   ├── IGroupRepository.ts
│   │   └── IExpenseRepository.ts
│   └── errors/                          # Erros de domínio
├── application/                         # Lógica de aplicação
│   ├── usecases/                       # Casos de uso (Use Cases)
│   │   ├── Register.ts                 # Registrar novo usuário
│   │   ├── Login.ts                   # Login de usuário
│   │   ├── CreateGroup.ts             # Criar novo grupo
│   │   ├── JoinGroup.ts               # Entrar em um grupo
│   │   ├── CreateExpense.ts           # Registrar despesa
│   │   ├── GetGroupBalances.ts        # Obter saldos do grupo
│   │   ├── GetUserActivities.ts       # Histórico do usuário
│   │   └── ... (mais use cases)
│   ├── infrastructure/                 # Implementação técnica
│   │   ├── database/                  # Configuração do banco
│   │   │   └── prismaClient.ts
│   │   ├── repositories/              # Implementações dos repositórios
│   │   │   ├── PrismaUserRepository.ts
│   │   │   ├── PrismaGroupRepository.ts
│   │   │   └── PrismaExpenseRepository.ts
│   ├── presentation/                  # Camada de apresentação
│   │   ├── controllers/               # Controllers (request/response)
│   │   │   ├── UserController.ts
│   │   │   ├── GroupController.ts
│   │   │   ├── ExpenseController.ts
│   │   │   ├── BalanceController.ts
│   │   │   └── ActivityController.ts
│   │   ├── routes/                   # Definição de rotas HTTP
│   │   │   ├── userRoutes.ts
│   │   │   ├── groupRoutes.ts
│   │   │   ├── expenseRoutes.ts
│   │   │   ├── balanceRoutes.ts
│   │   │   └── activityRoutes.ts
│   ├── shared/                        # Código compartilhado
│   │   ├── errors/                   # Classes de erro
│   │   ├── uploads/                 # Gestão de uploads
│   │   └── constants/               # Constantes globais
│   ├── app.ts                        # Configuração da aplicação Fastify
│   └── server.ts                     # Entrada da aplicação

```

2.3 Fluxo de Requisição


```
Cliente HTTP (Mobile App)
  ↓
Fastify Server
  ↓
Routes (presentation/routes/)
  ↓
Controllers (presentation/controllers/)
  ↓
Use Cases (application/usecases/)
  ↓
Repositories (infrastructure/repositories/)
  ↓
Prisma Client
  ↓
PostgreSQL Database
```

2.4 Injeção de Dependência

As dependências são injetadas no arquivo `app.ts`:

```
// Exemplo: Controllers recebem Use Cases como dependências
const userController = new UserController(
  new Register(userRepository),
  new Login(userRepository),
  // ... mais use cases
);
```

3. Banco de Dados

3.1 Schema do Banco de Dados

Entidades principais:

User (Usuários)

```
id (INT, PK)
name (VARCHAR)
email (VARCHAR, UNIQUE)
passwordHash (VARCHAR)
pixKey (VARCHAR, nullable)
resetCode (VARCHAR, nullable)
resetCodeExpiresAt (DATETIME, nullable)
createdAt (DATETIME)

Relacionamentos:
- 1:N com GroupMember (participação em grupos)
- 1:N com Group (grupos criados)
- 1:N com Expense (despesas pagas)
- 1:N com ExpenseSplit (divisões de despesa)
```

Group (Grupos)

```
id (INT, PK)
name (VARCHAR)
category (VARCHAR)
inviteToken (VARCHAR, UNIQUE)
createdByUserId (INT, FK)
createdAt (DATETIME)

Relacionamentos:
- N:1 com User (criador)
- N:N com User via GroupMember (membros)
- 1:N com Expense (despesas do grupo)
```

GroupMember (Membros do Grupo)

```
groupId (INT, PK, FK)
userId (INT, PK, FK)
joinedAt (DATETIME)

PK Composto: (groupId, userId)
Cascade Delete: ao deletar Group ou User
```

Expense (Despesas)

```
id (INT, PK)
groupId (INT, FK)
paidByUserId (INT, FK)
amount (DECIMAL)
description (VARCHAR)
receiptUrl (VARCHAR, nullable)
createdAt (DATETIME)

Relacionamentos:
- N:1 com Group
- N:1 com User (quem pagou)
- 1:N com ExpenseSplit (divisão da despesa)
```

ExpenseSplit (Divisões de Despesa)

```
id (INT, PK)
expenseId (INT, FK)
userId (INT, FK)
amountOwed (DECIMAL)
createdAt (DATETIME)

Relacionamentos:
- N:1 com Expense
- N:1 com User
```

3.2 Características do Prisma

- **ORM Type-Safe:** Geração automática de tipos TypeScript
- **Migrações:** Controle de versão do schema via arquivos
- **Seed Data:** Possibilidade de popular dados iniciais
- **Adapter PostgreSQL:** Uso de `@prisma/adaptor-pg` para otimização

4. Camadas da API em Detalhe

4.1 Camada de Domínio (Domain Layer)

Responsabilidade: Definir as regras de negócio e contratos

Entities (Entidades):

- Representam objetos do domínio
- Contêm validações específicas de negócio
- Exemplo: User, Group, Expense

Repositories (Interfaces):

- Definem contratos de acesso a dados
- Permitem abstração da implementação
- Exemplo: IUserRepository, IGroupRepository

4.2 Camada de Aplicação (Application Layer)

Responsabilidade: Orquestrar a lógica de negócio através de Use Cases

Use Cases:

- Cada Use Case representa uma ação do sistema
- Recebem repositórios como dependências
- Validam entrada e executam operações
- Retomam resultados ou lançam erros

Use Cases Principais:

1. Autenticação

- Register.ts - Registrar novo usuário
- Login.ts - Autenticar usuário
- ForgotPassword.ts - Solicitar reset
- ResetPassword.ts - Confirmar reset
- LinkAccount.ts - Vincular conta social

2. Grupos

- CreateGroup.ts - Criar novo grupo
- JoinGroup.ts - Entrar em grupo
- DeleteGroup.ts - Deletar grupo
- GetGroupByInvite.ts - Buscar por link de convite
- GetUserGroups.ts - Listar grupos do usuário

3. Despesas

- CreateExpense.ts - Registrar nova despesa
- GetGroupActivities.ts - Histórico de despesas

4. Saldos e Resumos

- GetGroupBalances.ts - Calcular quem deve para quem
- GetUserSummary.ts - Resumo financeiro do usuário
- GetUserActivities.ts - Histórico de atividades

5. Perfil

- UpdatePixKey.ts - Atualizar chave PIX

4.3 Camada de Infraestrutura (Infrastructure Layer)

Responsabilidade: Implementação técnica de repositórios e serviços externos

Prisma Client:

```
// infrastructure/database/prismaClient.ts
import { PrismaClient } from "@prisma/client";

export const prisma = new PrismaClient();
```

Repositórios Implementados:

```
// infrastructure/repositories/PrismaUserRepository.ts
export class PrismaUserRepository implements IUserRepository {
  async findById(id: number): Promise<User | null> {
    return prisma.user.findUnique({ where: { id } });
  }

  async findByEmail(email: string): Promise<User | null> {
    return prisma.user.findUnique({ where: { email } });
  }

  async create(data: CreateUserInput): Promise<User> {
    return prisma.user.create({ data });
  }
}
```

4.4 Camada de Apresentação (Presentation Layer)

Responsabilidade: Receber requisições HTTP e enviar respostas

Controllers:

- Validam parâmetros de entrada
- Chamam Use Cases
- Formatam e retornam respostas

Routes:

- Definem endpoints HTTP
- Mapeiam requisições para controladores
- Tratamento de erros global

5. Arquitetura do Aplicativo Mobile

5.1 Stack Tecnológico

- **React Native** - Framework de desenvolvimento mobile
- **Expo** - Plataforma que simplifica React Native (build, deployment)
- **React Navigation** - Sistema de navegação entre telas
- **TypeScript** - Tipagem estática

5.2 Estrutura do Projeto

```
src/
├── assets/                # Imagens, fontes, recursos estáticos
├── components/           # Componentes reutilizáveis
│   ├── FloatingLabelInput.tsx
│   ├── InputField.tsx
│   ├── PrimaryButton.tsx
│   ├── ProgressBar.tsx
│   ├── Skeleton.tsx
│   └── ...
├── icons/               # Componentes de ícones SVG
│   ├── AppleIcon.tsx
│   ├── GoogleIcon.tsx
│   ├── CheckIcon.tsx
│   ├── LockIcon.tsx
│   ├── LogoMePaga.tsx
│   └── ...
├── screens/             # Telas da aplicação (16 telas)
│   ├── LoginScreen.tsx
│   ├── CreateGroupScreen.tsx
│   ├── CreateExpenseScreen.tsx
│   ├── GroupDetailScreen.tsx
│   ├── ProfileScreen.tsx
│   ├── RegisterStep1Screen.tsx
│   ├── RegisterStep2Screen.tsx
│   ├── RegisterStep3Screen.tsx
│   ├── RegisterSuccessScreen.tsx
│   ├── ForgotStep1Screen.tsx
│   ├── ForgotStep2Screen.tsx
│   ├── ForgotStep3Screen.tsx
│   ├── ForgotSuccessScreen.tsx
│   └── ...
├── services/           # Serviços (API, storage, etc)
│   ├── api.ts          # Cliente HTTP para backend
│   └── ...
├── navigation.ts        # Definição de tipos e estrutura de navegação
├── theme.ts             # Temas, cores, estilos globais
└── App.tsx             # Componente raiz da aplicação
```

5.3 Sistema de Navegação

O aplicativo usa **React Navigation** com uma pilha de navegação (Stack Navigation):

```
// navigation.ts
export type RootStackParamList = {
  // Autenticação
  Login: undefined;
  RegisterStep1: undefined;
  RegisterStep2: { nome: string; sobrenome: string; email: string };
  RegisterStep3: { /* dados necessários */ };
  RegisterSuccess: undefined;
  ForgotStep1: undefined;
  ForgotStep2: { email: string };
  ForgotStep3: { email: string; code: string };
  ForgotSuccess: undefined;

  // Home e Perfil
  Home: { userId: number; name: string; email: string; pixKey?: string };
  Profile: { userId: number; /* ... */ };
  ProfilePixKey: { userId: number; pixKey?: string };

  // Grupos
  CreateGroup: { userId: number; name: string };
  GroupDetail: { userId: number; groupId: number; /* ... */ };
  GroupSettings: { userId: number; groupId: number; /* ... */ };

  // Despesas
  CreateExpense: { userId: number; groupId: number; /* ... */ };
};
```

6. Fluxos Principais da Aplicação

6.1 Fluxo de Autenticação - Registro

```
RegisterStep1Screen
  ↓ (Insere nome, sobrenome)
RegisterStep2Screen
  ↓ (Insere email, validação)
RegisterStep3Screen
  ↓ (Insere chave PIX)
Register (UseCase)
  ├── Validar email duplicado
  ├── Hash de senha (bcrypt)
  └── Criar usuário no banco
      ↓
RegisterSuccessScreen
  ↓
Login
```

6.2 Fluxo de Login

```
LoginScreen (email + senha)
  ↓
Login UseCase
  ├── Buscar usuário por email
  ├── Comparar senha com hash
  └── Retornar dados do usuário
      ↓
HomeScreen
```

6.3 Fluxo de Criar Grupo

```

HomeScreen (botão "Novo Grupo")
  ↓
CreateGroupScreen (nome + categoria)
  ↓
CreateGroup UseCase
  ├── Validar dados
  ├── Gerar token único de convite
  ├── Criar grupo no banco
  └── Adicionar criador como membro
    ↓
GroupDetailScreen

```

6.4 Fluxo de Criar Despesa

```

GroupDetailScreen (botão "Adicionar Despesa")
  ↓
CreateExpenseScreen (seleção de membros + valores)
  ↓
CreateExpense UseCase
  ├── Validar dados
  ├── Criar registro de Expense
  ├── Criar ExpenseSplits (divisão para cada membro)
  └── Persistir no banco
    ↓
GroupDetailScreen (atualizado com despesa)

```

6.5 Fluxo de Visualizar Saldos

```

GroupDetailScreen
  ↓
GetGroupBalances UseCase
  ├── Calcular total gasto por membro (paidBy)
  ├── Calcular total devido por membro (ExpenseSplit)
  ├── Calcular líquido (crédito/débito)
  └── Retornar quem deve para quem
    ↓
BalanceScreen (exibição de saldos)

```

7. Endpoints da API

7.1 Autenticação

```

POST /auth/register
Body: { name, email, password, pix_key? }
Response: 201 { id, name, email, is_temporary }

POST /auth/login
Body: { email, password }
Response: 200 { id, name, email, pix_key?, is_temporary }

POST /auth/forgot-password
Body: { email }
Response: 200 { message }

POST /auth/reset-password
Body: { email, code, new_password }
Response: 200 { message }

POST /auth/link-account
Body: { user_id, email }
Response: 200 { id, name, email, is_temporary }

```

7.2 Grupos

```
POST /groups
Body: { name, category, created_by_user_id }
Response: 201 { id, name, category, invite_token, created_at }

POST /groups/join
Body: { invite_token, user_id }
Response: 200 { message }

GET /groups/:groupId/invite
Response: 200 { group, members }

GET /users/:userId/groups
Response: 200 [{ id, name, category, memberCount, ... }]

DELETE /groups/:groupId
Response: 200 { message }
```

7.3 Despesas

```
POST /expenses
Body: { group_id, paid_by_user_id, amount, description, receipt_url? }
Response: 201 { id, group_id, paid_by_user_id, amount, ... }

GET /groups/:groupId/activities
Response: 200 [{ type, user, amount, description, created_at, ... }]

GET /users/:userId/activities
Response: 200 [{ type, group, amount, description, created_at, ... }]
```

7.4 Saldos

```
GET /groups/:groupId/balances
Response: 200 {
  total: Decimal,
  balances: [
    { user_id, name, paidTotal, owedTotal, balance },
    ...
  ],
  settlements: [
    { from_user, to_user, amount },
    ...
  ]
}
```

7.5 Perfil

```
PATCH /users/:userId/pix-key
Body: { pix_key }
Response: 200 { id, name, email, pix_key }

GET /users/:userId/summary
Response: 200 { id, name, email, friendCount, groupCount, ... }
```

REQUISITOS - ANÁLISE COMPLETA

1. Visão Geral do Produto

1.1 Descrição do Produto

MePaga é uma aplicação móvel que simplifica o gerenciamento de despesas compartilhadas entre grupos (amigos, colegas, familiares). Permite que usuários criem grupos, registrem despesas e saibam automaticamente quanto cada pessoa deve pagar ou receber.

1.2 Problema que Resolve

- Dificuldade em lembrar quem deve quanto a quem
- Falta de transparência em divisão de custos

- Necessidade de cálculos manuais complexos
- Dificuldade em rastrear transações de compensação

1.3 Proposta de Valor

- ☒ **Simplicidade:** Interface intuitiva e fácil de usar
- ☒ **Precisão:** Cálculos automáticos de saldos
- ☒ **Transparência:** Histórico completo de transações
- ☒ **Integração PIX:** Chave PIX para facilitar transferências
- ☒ **Multi-plataforma:** Funciona em iOS, Android e Web

2. Requisitos Funcionais

2.1 Gerenciamento de Usuários

RF2.1.1 Autenticação

- **Descrição:** O sistema deve permitir que usuários se autenticem na aplicação
- **Atores:** Usuário não autenticado
- **Pré-condições:** Usuário possui conta ativa
- **Fluxo Principal:**
 1. Usuário insere email e senha
 2. Sistema valida credenciais contra banco de dados
 3. Se válido, autentica usuário e permite acesso
 4. Se inválido, exibe mensagem de erro
- **Pós-condições:** Usuário logado tem acesso à home
- **Restrições:** Limite de tentativas de login
- **Prioridade:** CRÍTICA

RF2.1.2 Registro de Novo Usuário

- **Descrição:** Novos usuários podem se registrar na plataforma
- **Atores:** Usuário não registrado
- **Fluxo Principal:**
 1. Usuário acessa tela de registro
 2. Insere nome, sobrenome, email
 3. Insere senha
 4. Insere chave PIX (opcional no registro)
 5. Sistema valida dados
 6. Sistema cria novo usuário
 7. Exibe mensagem de sucesso
- **Validações:**
 - Email deve ser único (não duplicado)
 - Email deve estar em formato válido
 - Senha deve ter mínimo de 8 caracteres
 - Todos os campos obrigatórios devem ser preenchidos
- **Pós-condições:** Novo usuário criado, pode fazer login
- **Prioridade:** CRÍTICA

RF2.1.3 Recuperação de Senha

- **Descrição:** Usuários podem recuperar acesso ao fazer reset de senha
- **Atores:** Usuário que esqueceu senha
- **Fluxo Principal:**
 1. Usuário clica em "Esqueci a senha"
 2. Insere email registrado
 3. Sistema envia código de 6 dígitos para email
 4. Usuário insere código recebido
 5. Usuário define nova senha
 6. Sistema valida e atualiza senha no banco
 7. Usuário pode fazer login com nova senha
- **Validações:**

- Email deve estar registrado no sistema
 - Código deve ser válido e não expirado (15 minutos)
 - Nova senha deve atender requisitos mínimos
- **Pós-condições:** Senha alterada com sucesso
- **Prioridade:** ALTA

RF2.1.4 Atualizar Chave PIX

- **Descrição:** Usuário pode adicionar ou atualizar sua chave PIX
- **Atores:** Usuário autenticado
- **Fluxo Principal:**
 1. Usuário acessa perfil
 2. Clica em "Editar chave PIX"
 3. Insere/atualiza chave PIX
 4. Sistema salva a alteração
 5. Exibe confirmação
- **Validações:**
 - Chave PIX deve estar em formato válido
- **Pós-condições:** Chave PIX atualizada
- **Prioridade:** MÉDIA

2.2 Gerenciamento de Grupos

RF2.2.1 Criar Grupo

- **Descrição:** Usuário autenticado pode criar um novo grupo
- **Atores:** Usuário autenticado
- **Fluxo Principal:**
 1. Usuário clica em "Novo Grupo"
 2. Insere nome do grupo
 3. Seleciona categoria (viagem, colegas, família, outro)
 4. Sistema cria grupo com token único de convite
 5. Usuário é adicionado como membro criador
 6. Exibe tela de sucesso com opção de convidar
- **Validações:**
 - Nome do grupo obrigatório (máximo 100 caracteres)
 - Categoria obrigatória
- **Pós-condições:** Novo grupo criado, usuário é membro
- **Prioridade:** CRÍTICA

RF2.2.2 Entrar em Grupo (Join)

- **Descrição:** Usuário pode entrar em um grupo via token de convite
- **Atores:** Usuário autenticado
- **Fluxo Principal:**
 1. Usuário clica em "Entrar em Grupo"
 2. Insere token de convite (ou acessa link)
 3. Sistema valida token
 4. Adiciona usuário como membro do grupo
 5. Exibe grupo na lista de grupos do usuário
- **Validações:**
 - Token deve ser válido
 - Token deve corresponder a um grupo existente
 - Usuário não pode ser membro duplicado
- **Pós-condições:** Usuário adicionado ao grupo
- **Prioridade:** CRÍTICA

RF2.2.3 Listar Grupos do Usuário

- **Descrição:** Exibir todos os grupos dos quais o usuário é membro
- **Atores:** Usuário autenticado
- **Dados Exibidos** por grupo:
 - Nome do grupo
 - Categoria
 - Número de membros
 - Saldo do usuário naquele grupo (simplificado)
 - Data de entrada no grupo
- **Ações Disponíveis:**
 - Clicar para ver detalhes
 - Sair do grupo (futuro)
- **Prioridade:** CRÍTICA

RF2.2.4 Visualizar Detalhes do Grupo

- **Descrição:** Exibir informações completas e atividades do grupo
- **Atores:** Membro do grupo
- **Dados Exibidos:**
 - Nome, categoria
 - Lista de membros com seus saldos
 - Histórico de despesas (últimas transações)
 - Total gasto no grupo
- **Ações Disponíveis:**
 - Adicionar despesa
 - Ver saldos (quem deve para quem)
 - Compartilhar convite
 - Configurações (se criador)
- **Prioridade:** CRÍTICA

2.3 Gerenciamento de Despesas

RF2.3.1 Criar Despesa

- **Descrição:** Membro do grupo pode registrar uma despesa
- **Atores:** Membro autenticado
- **Fluxo Principal:**
 1. Membro clica em "Adicionar Despesa"
 2. Insere descrição da despesa
 3. Insere valor total
 4. Seleciona quem pagou (padrão: usuário logado)
 5. Seleciona quem participa da despesa (checkboxes)
 6. Sistema calcula divisão (valores por pessoa)
 7. Permite upload de recibo (foto)
 8. Sistema cria registros de Expense e ExpenseSplit
 9. Exibe confirmação
- **Validações:**
 - Valor deve ser positivo
 - Deve haver pelo menos 1 pessoa na divisão
 - Descrição obrigatória (máx 255 caracteres)
 - Arquivo de recibo máximo 10 MB
- **Tipos de Divisão:**
 - Igualmente (divide em partes iguais)
 - Manualmente (insere valor para cada pessoa)
 - Percentual (futuro)
- **Pós-condições:** Despesa criada, ExpenseSplits geradas
- **Prioridade:** CRÍTICA

2.4 Visualização de Saldos

RF2.4.1 Visualizar Saldos do Grupo

- **Descrição:** Mostrar quem deve quanto para quem no grupo
- **Atores:** Membro do grupo
- **Fluxo Principal:**
 1. Membro clica em "Saldos" na tela de detalhes do grupo
 2. Sistema calcula:
 - Quanto cada pessoa pagou
 - Quanto cada pessoa deve (ExpenseSplits)
 - Saldo líquido (crédito ou débito)
 3. Exibe lista clara de liquidações necessárias
- **Formato de Exibição:**

João deve R\$ 150 para Maria
Maria deve R\$ 50 para Pedro
etc.

- **Prioridade:** CRÍTICA

3. Requisitos Não-Funcionais

3.1 Performance

Requisito	Especificação
Tempo de Resposta	APIs devem responder em < 500ms em condições normais
Carregamento de Tela	Telas devem carregar em < 2 segundos
Sincronização	Dados devem sincronizar entre dispositivos em < 5 segundos
Cálculo de Saldos	Cálculo de saldos até 1000 membros em < 1 segundo

3.2 Confiabilidade

Requisito	Especificação
Uptime	API deve ter uptime de 99.5%
Recuperação de Erro	Sistema deve recuperar de falhas de rede automaticamente
Integridade de Dados	Nenhuma perda de dados em falhas (transações ACID)
Backup	Backup automático do banco de dados a cada 24h

3.3 Usabilidade

Requisito	Especificação
Interface	Intuitiva e fácil de usar para usuários não-técnicos
Acessibilidade	Suporte a leitores de tela (futuro)
Responsive	Funciona em diferentes tamanhos de tela
Idioma	Português do Brasil (futuro: multilíngue)

3.4 Segurança

Requisito	Especificação
Senhas	Hash com bcrypt (no mínimo 10 rounds)
HTTPS	Todas as comunicações via HTTPS em produção
Validação	Validação rigorosa de todas as entradas
Autorização	Usuários só veem/acessam seus próprios dados
CORS	Apenas origens autorizadas podem acessar API
Rate Limiting	Limite de requisições por IP (futuro)

DOCUMENTACAO - GUIA CENTRAL

1. Como Usar Esta Documentação

Para Desenvolvedores Iniciando no Projeto

1. **Primeiro:** Leia ARQUITETURA.md seção 1-3
2. **Depois:** Explore as camadas (seção 4) para entender o padrão
3. **Depois:** Leia REQUISITOS.md seção 2 para entender funcionalidades
4. **Por fim:** Use os endpoints (ARQUITETURA.md seção 7) como referência

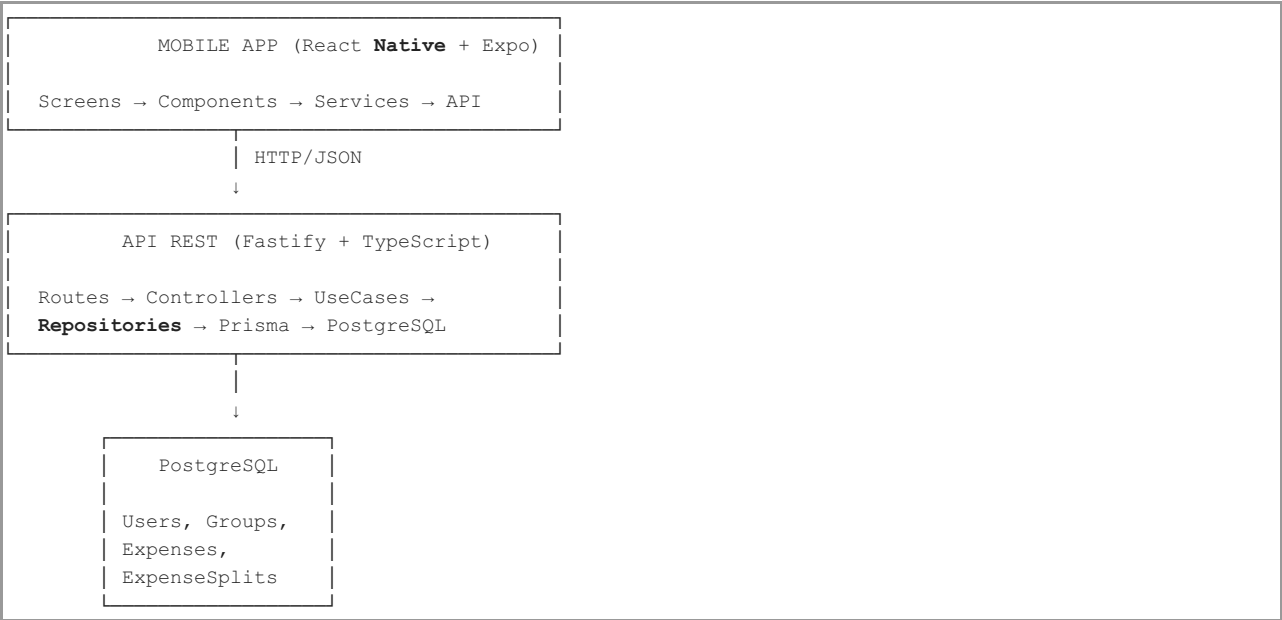
Para Product Managers / Stakeholders

1. **Primeiro:** Leia REQUISITOS.md seção 1-3
2. **Depois:** Revise os casos de uso (seção 6) e user stories (seção 7)
3. **Consulte:** Roadmap futuro (seção 9) para planejamento
4. **Use:** Matriz de rastreamento (seção 10) para status das funcionalidades

Para Testadores / QA

1. **Primeiro:** Leia REQUISITOS.md seção 2 (Requisitos Funcionais)
2. **Depois:** Revise critérios de aceitação em cada funcionalidade
3. **Consulte:** Fluxos de dados em ARQUITETURA.md seção 6
4. **Use:** User stories para casos de teste

2. Resumo da Arquitetura



QUICK_REFERENCE - CHEAT SHEET

Comandos Essenciais

Backend (mepaga-api)

```
# Desenvolvimento
pnpm dev           # Start dev server com watch
pnpm build         # Compilar TypeScript para JS
pnpm start         # Rodar versão compilada

# Banco de dados
pnpm prisma:migrate # Criar/rodar migrations
pnpm prisma:generate # Gerar tipos Prisma
pnpm prisma:studio  # Abrir Prisma Studio
```

Frontend (mepaga-app)

```
# Desenvolvimento
pnpm start          # Iniciar Expo dev server
pnpm android        # Build para Android
pnpm ios            # Build para iOS
pnpm web            # Rodar em web
pnpm reverse         # Forward port Android
```

Estrutura Rápida

Backend

```
src/
├─ domain/entities/*.ts    # Modelos
├─ domain/repositories/*.ts # Interfaces
├─ application/usecases/*.ts # Lógica
├─ infrastructure/repositories/*.ts # Implementação
├─ presentation/controllers/*.ts # Controllers
├─ presentation/routes/*.ts # Rotas
└─ shared/ # Erros, utils
```

Frontend

```
src/
├─ screens/*.tsx    # Telas (16)
├─ components/*.tsx # Componentes
├─ icons/*.tsx      # Ícones SVG
├─ services/api.ts  # Client HTTP
├─ navigation.ts    # Tipos
└─ theme.ts         # Estilos
```

Como Criar Um Use Case

```

// 1. Entity
export class MyEntity {
  constructor(public id: number) {}
}

// 2. Repository Interface
export interface IMyRepository {
  create(data: Input): Promise<MyEntity>;
}

// 3. Use Case
export class MyUseCase {
  constructor(private repo: IMyRepository) {}
  async execute(input: Input): Promise<MyEntity> {
    return this.repo.create(input);
  }
}

// 4. Repository Implementation
export class PrismaMyRepository implements IMyRepository {
  async create(data: Input) {
    return prisma.my.create({ data });
  }
}

// 5. Controller
export class MyController {
  constructor(private useCase: MyUseCase) {}
  async create(req: FastifyRequest, reply: FastifyReply) {
    const result = await this.useCase.execute(req.body);
    return reply.status(201).send(result);
  }
}

// 6. Routes
export async function myRoutes(app: FastifyInstance) {
  app.post('/my', (req, reply) => myController.create(req, reply));
}

// 7. Em app.ts
const myUseCase = new MyUseCase(new PrismaMyRepository());
const myController = new MyController(myUseCase);

```

Testes Rápidos

```
# Registrar
curl -X POST http://localhost:3000/auth/register \
-H "Content-Type: application/json" \
-d '{
  "name": "João",
  "email": "joao@example.com",
  "password": "senha123456"
}'

# Login
curl -X POST http://localhost:3000/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "joao@example.com",
  "password": "senha123456"
}'

# Criar Grupo
curl -X POST http://localhost:3000/groups \
-H "Content-Type: application/json" \
-d '{
  "name": "Viagem Fortaleza",
  "category": "viagem",
  "created_by_user_id": 1
}'

# Ver Saldos
curl -X GET http://localhost:3000/groups/1/balances
```

Documentação Completa - v1.0.0

Última atualização: 2026-06-11